In [26]:

```
# RA1 - Ingestão e Preparação | Repasses da Arrecadação Federal
from pathlib import Path
import urllib.request
import pandas as pd

pd.set_option("display.max_columns", None)
pd.set_option("display.float_format", "{:,.2f}".format)

# Caminhos (funciona com o notebook em notebooks/ ou na raiz do projeto / Colab)
BASE = Path("..") if Path("../data").exists() else Path(".")
RAW = BASE / "data" / "raw" / "repasses.csv"
PROCESSED = BASE / "data" / "processed" / "repasses_arrecadacao_federal_limpo.csv"
RAW.parent.mkdir(parents=True, exist_ok=True)
PROCESSED.parent.mkdir(parents=True, exist_ok=True)

# Fonte oficial: https://dados.gov.br/dados/conjuntos-dados/repasses-da-arrecadacao-federal
# Cópia do arquivo usada no projeto (Drive):
DRIVE_ID = "1lLxFUzdouLUESomjn0tqO_jjOiG2V_UT"
URL_DRIVE = f"https://drive.google.com/uc?export=download&id={DRIVE_ID}"

COL_DATA = "Mês / Ano de Referência"
COL_ENT = "Entidade"
COL_COD = "UC/CNPJ"
COL_VAL = "Total Repassado"
CHAVE = [COL_DATA, COL_ENT]          # um repasse por mês e entidade

MESES = {"jan": 1, "fev": 2, "mar": 3, "abr": 4, "mai": 5, "jun": 6,
         "jul": 7, "ago": 8, "set": 9, "out": 10, "nov": 11, "dez": 12}


def converte_valor(s, errors="raise"):
    """'2.604.077.590,00' e '252386984,9' → float (ponto = milhar, vírgula = decimal)."""
    return pd.to_numeric(
        s.str.replace(".", "", regex=False).str.replace(",", ".", regex=False),
        errors=errors,
    )


def fora_iqr(s):
    """Marca valores fora de [Q1-1,5*IQR ; Q3+1,5*IQR]. Se IQR = 0, o método não se aplica."""
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    if iqr == 0:
        return pd.Series(False, index=s.index)
    return (s < q1 - 1.5 * iqr) | (s > q3 + 1.5 * iqr)
```

```
print("pandas", pd.__version__)
```

```
pandas 3.0.5
```

In [27]:
```
# 2. INGESTÃO
# 2.1 Obtenção: usa a cópia local em data/raw/; se não existir, baixa uma
vez do Drive.
if not RAW.exists():
    urllib.request.urlretrieve(URL_DRIVE, RAW)
    print("Arquivo baixado em", RAW)

# O Drive pode devolver uma página HTML em vez do CSV quando o download
falha
with open(RAW, "rb") as f:
    assert not f.read(200).lstrip().lower().startswith(b"<"), \
        "O arquivo baixado é HTML, não CSV. Baixe manualmente do
dados.gov.br e salve em " + str(RAW)
print("Arquivo bruto:", RAW)
```

```
Arquivo bruto: ..\data\raw\repasses.csv
```

In [28]:
```
# 2.2 Leitura: tudo como texto (dtype=str) para o pandas não adivinhar
tipos.
# A conversão de tipos é feita na limpeza (D4 e D5).
for enc in ("utf-8", "latin-1"):
    try:
        df = pd.read_csv(RAW, sep=";", dtype=str, encoding=enc)
        print("Leitura concluída | separador ';' | encoding:", enc)
        break
    except UnicodeDecodeError:
        continue

df_raw = df.copy()          # versão original, usada para o "antes" nas
comparações
```

```
Leitura concluída | separador ';' | encoding: utf-8
```

In [29]:
```
# 2.3 Verificação do carregamento
print("Linhas x colunas:", df_raw.shape)
print("Colunas:", list(df_raw.columns))
df_raw.info()
df_raw.head(10)
```

```
Linhas x colunas: (2397, 4)
Colunas: ['Mês / Ano de Referência', 'Entidade', 'UC/CNPJ', 'Total
Repassado']
<class 'pandas.DataFrame'>
RangeIndex: 2397 entries, 0 to 2396
Data columns (total 4 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Mês / Ano de Referência  2380 non-null   str
 1   Entidade                 2380 non-null   str
 2   UC/CNPJ                  2380 non-null   str
 3   Total Repassado          2380 non-null   str
dtypes: str(4)
memory usage: 75.0 KB
```

Out[29]:

| | Mês / Ano de Referência | Entidade | UC/CNPJ | Total Repassado |
|---|---|---|---|---|
| **0** | jan/15 | FNDE | 153173 / 15253 | 2.604.077.590,00 |
| **1** | jan/15 | INCRA | 373001 / 37201 | 181.456.016,00 |
| **2** | jan/15 | SENAI | 33.564.543/0001-90 | 219.119.330,00 |
| **3** | jan/15 | SESI | 33.641.358/0001-52 | 313.370.139,00 |
| **4** | jan/15 | SENAC | 33.469.172/0001-68 | 334.005.249,00 |
| **5** | jan/15 | SESC | 33.469.164/0001-11 | 597.447.338,00 |
| **6** | jan/15 | SEBRAE | 00.330.854/0001-45 | 428.247.534,00 |
| **7** | jan/15 | EMBRATUR | 35.842.428/0001-66 | 0 |
| **8** | jan/15 | DPC | 852001 / 00001 | 20.988.896,00 |
| **9** | jan/15 | ANAC | 122002 / 00001 | 30.221.162,00 |

In [30]:
```
# 3. COMPREENSÃO E QUALIDADE (somente diagnóstico; df_raw não é alterado)
# Unidade de observação: uma linha = valor repassado em um mês a uma
entidade.

# Q1 - Valores ausentes
vazias = df_raw.isna().all(axis=1)
parciais = df_raw.isna().any(axis=1) & ~vazias
print("Linhas totalmente vazias:", vazias.sum())
print("Linhas parcialmente vazias:", parciais.sum())
print("Total de células nulas:", int(df_raw.isna().sum().sum()))
print("Posição das linhas vazias:", df_raw.index[vazias].min(), "a",
df_raw.index[vazias].max())
display(df_raw.isna().sum().to_frame("nulos"))

# Base de trabalho do diagnóstico: apenas linhas com dados
base = df_raw.dropna().copy()
print("Linhas com dados:", len(base))
```

```
Linhas totalmente vazias: 17
Linhas parcialmente vazias: 0
Total de células nulas: 68
Posição das linhas vazias: 2380 a 2396
```

| | nulos |
|---|---|
| **Mês / Ano de Referência** | 17 |
| **Entidade** | 17 |
| **UC/CNPJ** | 17 |
| **Total Repassado** | 17 |

```
Linhas com dados: 2380
```

In [31]:
```
# Q2 e Q3 - Registros duplicados
print("Q2 - Linhas idênticas (todas as linhas):", df_raw.duplicated().sum())
print("Q2 - Linhas idênticas entre as linhas com dados:",
base.duplicated().sum())

print("Q3 - Duplicados por Mês + Entidade (valores brutos):",
base.duplicated(subset=CHAVE).sum())
ent_pad = base[COL_ENT].str.strip().replace({"APEX": "APEX-BR"})
chave_pad = pd.concat([base[COL_DATA].str.strip(), ent_pad], axis=1)
print("Q3 - Duplicados por Mês + Entidade (após strip e APEX → APEX-BR):",
chave_pad.duplicated().sum())

base[base.duplicated(subset=CHAVE, keep=False)].sort_values(CHAVE)
```

```
Q2 - Linhas idênticas (todas as linhas): 16
Q2 - Linhas idênticas entre as linhas com dados: 0
Q3 - Duplicados por Mês + Entidade (valores brutos): 0
Q3 - Duplicados por Mês + Entidade (após strip e APEX → APEX-BR): 0
```

Out[31]:

| | Mês / Ano de Referência | Entidade | UC/CNPJ | Total Repassado |
|---|---|---|---|---|

In [32]:
```
# Q4 - Tipos de dados incorretos
display(df_raw.dtypes.to_frame("tipo lido"))

fmt_data = base[COL_DATA].str.fullmatch(r"[a-z]{3}/\d{2}")
print("Datas fora do padrão 'mmm/aa':", (~fmt_data).sum())
print("Meses desconhecidos:",
(~base[COL_DATA].str[:3].isin(list(MESES))).sum())
print("Conclusão: data e valor foram lidos como texto e precisam ser
convertidos.")
base[COL_DATA].head()
```

| | tipo lido |
|---|---|
| **Mês / Ano de Referência** | str |
| **Entidade** | str |
| **UC/CNPJ** | str |
| **Total Repassado** | str |

```
Datas fora do padrão 'mmm/aa': 0
Meses desconhecidos: 0
Conclusão: data e valor foram lidos como texto e precisam ser convertidos.
```

Out[32]:
```
0    jan/15
1    jan/15
2    jan/15
3    jan/15
4    jan/15
Name: Mês / Ano de Referência, dtype: str
```

In [33]:
```
# Q5 - Formatos inconsistentes do valor (Total Repassado)
v = base[COL_VAL]
com_milhar = v.str.contains(r"\.\d{3}")
zeros_txt = v == "0"
sem_milhar = ~com_milhar & ~zeros_txt

print("Valores com separador de milhar:", com_milhar.sum())
print("Valores sem separador de milhar:", sem_milhar.sum())
print("Valores '0':", zeros_txt.sum())
print("Meses com valor sem milhar:", base.loc[sem_milhar,
COL_DATA].unique())
print("Sem milhar contendo ponto (deve ser 0):",
v[sem_milhar].str.contains(r"\.").sum())
print("Valores com vírgula decimal:", v.str.contains(",").sum())
base[sem_milhar].head(10)
```

```
Valores com separador de milhar: 2070
Valores sem separador de milhar: 176
Valores '0': 134
Meses com valor sem milhar: <StringArray>
['out/25', 'nov/25', 'dez/25', 'jan/26', 'fev/26', 'mar/26', 'abr/26',
 'mai/26', 'jun/26', 'jul/26', 'ago/26']
Length: 11, dtype: str
Sem milhar contendo ponto (deve ser 0): 0
Valores com vírgula decimal: 2223
```

Out[33]:

| | Mês / Ano de Referência | Entidade | UC/CNPJ | Total Repassado |
|---|---|---|---|---|
| **2193** | out/25 | FNDE | 153173 / 15253 | 2950662651 |
| **2194** | out/25 | INCRA | 373001 / 37201 | 252386984,9 |
| **2195** | out/25 | SENAI | 33.564.543/0001-90 | 218030840,1 |
| **2196** | out/25 | SESI | 33.641.358/0001-52 | 319516639,5 |
| **2197** | out/25 | SENAC | 33.469.172/0001-68 | 429318140,3 |
| **2198** | out/25 | SESC | 33.469.164/0001-11 | 739128752,9 |
| **2199** | out/25 | SEBRAE | 00.330.854/0001-45 | 472771291,6 |
| **2201** | out/25 | DPC | 852001 / 00001 | 23184616,26 |
| **2202** | out/25 | ANAC | 122002 / 00001 | 24891430,8 |
| **2203** | out/25 | SENAR | 37.138.245/0001-90 | 215406046,8 |

In [34]:
```
# Q6 - Categorias inconsistentes (Entidade)
print([repr(e) for e in base[COL_ENT].unique()])
display(base[COL_ENT].value_counts())

# Cada entidade tem um único identificador?
display(base.groupby(base[COL_ENT].str.strip())[COL_COD].unique())

# APEX x APEX-BR: mesmo identificador e períodos que não se sobrepõem?
apex = base[base[COL_ENT].isin(["APEX", "APEX-BR"])]
apex.groupby(COL_ENT).agg(
    primeiro_mes=(COL_DATA, "first"),
    ultimo_mes=(COL_DATA, "last"),
    registros=(COL_DATA, "count"),
    identificador=(COL_COD, "unique"),
)
```

```
["'FNDE'", "'INCRA'", "'SENAI'", "'SESI'", "'SENAC'", "'SESC'", "'SEBRAE'",
"'EMBRATUR'", "'DPC'", "'ANAC'", "'SENAR'", "'SEST'", "'SENAT'",
"'SESCOOP'", "'SDR'", "'APEX-BR'", "'ABDI'", "'FNDE '", "'APEX'"]
```

```
Entidade
INCRA       140
SENAI       140
SESI        140
SENAC       140
SESC        140
SEBRAE      140
EMBRATUR    140
DPC         140
ANAC        140
SENAR       140
SEST        140
SENAT       140
SESCOOP     140
SDR         140
ABDI        140
FNDE        137
APEX-BR     132
APEX          8
FNDE          3
Name: count, dtype: int64
```

```
Entidade
ABDI        [07.200.966/0001-11]
ANAC            [122002 / 00001]
APEX        [05.507.500/0001-38]
APEX-BR     [05.507.500/0001-38]
DPC             [852001 / 00001]
EMBRATUR    [35.842.428/0001-66]
FNDE            [153173 / 15253]
INCRA           [373001 / 37201]
SDR             [420013 / 00001]
SEBRAE      [00.330.854/0001-45]
```

```
SENAC       [33.469.172/0001-68]
SENAI       [33.564.543/0001-90]
SENAR       [37.138.245/0001-90]
SENAT       [73.471.963/0001-47]
SESC        [33.469.164/0001-11]
SESCOOP     [03.087.543/0001-86]
SESI        [33.641.358/0001-52]
SEST        [73.471.989/0001-95]
Name: UC/CNPJ, dtype: object
```

Out[34]:

| | primeiro_mes | ultimo_mes | registros | identificador |
|---|---|---|---|---|
| **Entidade** | | | | |
| **APEX** | jan/26 | ago/26 | 8 | [05.507.500/0001-38] |
| **APEX-BR** | jan/15 | dez/25 | 132 | [05.507.500/0001-38] |

In [35]:

```
# Q7 - Formato do identificador (UC/CNPJ)
cod = base[COL_COD].str.strip()
e_cnpj = cod.str.fullmatch(r"\d{2}\.\d{3}\.\d{3}/\d{4}-\d{2}")
e_uc = cod.str.fullmatch(r"\d+ / \d+")
print("Formato CNPJ (00.000.000/0000-00):", e_cnpj.sum())
print("Formato 'NNNNNN / NNNNN':", e_uc.sum())
print("Fora dos dois padrões:", (~e_cnpj & ~e_uc).sum())
cod[~e_cnpj & ~e_uc].value_counts()
```

```
Formato CNPJ (00.000.000/0000-00): 1680
Formato 'NNNNNN / NNNNN': 700
Fora dos dois padrões: 0
```

Out[35]:

```
Series([], Name: count, dtype: int64)
```

In [36]:
```
# Q8 - Valores inválidos ou impossíveis
valor = converte_valor(base[COL_VAL], errors="coerce")
print("Valores que falharam na conversão:", valor.isna().sum())
print("Valores negativos:", (valor < 0).sum())
print("Valores zerados:", (valor == 0).sum())
display(base[valor < 0])

# Contexto do negativo: meses vizinhos da mesma entidade
neg = base[valor < 0]
for _, r in neg.iterrows():
    da_entidade = base[base[COL_ENT] == r[COL_ENT]].assign(Valor=valor)
    pos = da_entidade.index.get_loc(r.name)
    display(da_entidade.iloc[max(pos - 2, 0): pos + 3])

# Contexto dos zeros: quais entidades e quando há valor diferente de zero
display(base.loc[valor == 0, COL_ENT].value_counts())
base[base[COL_ENT] == "EMBRATUR"].assign(Valor=valor).query("Valor != 0")
```

```
Valores que falharam na conversão:

 0
Valores negativos: 1
Valores zerados: 134
```

| | Mês / Ano de Referência | Entidade | UC/CNPJ | Total Repassado |
|---|---|---|---|---|
| **2049** | jan/25 | ANAC | 122002 / 00001 | -318.467.181,81 |

| | Mês / Ano de Referência | Entidade | UC/CNPJ | Total Repassado | Valor |
|---|---|---|---|---|---|
| **2015** | nov/24 | ANAC | 122002 / 00001 | 18.501.620,88 | 18,501,620.88 |
| **2032** | dez/24 | ANAC | 122002 / 00001 | 22.501.059,66 | 22,501,059.66 |
| **2049** | jan/25 | ANAC | 122002 / 00001 | -318.467.181,81 | -318,467,181.81 |
| **2066** | fev/25 | ANAC | 122002 / 00001 | 14.565.849,20 | 14,565,849.20 |
| **2083** | mar/25 | ANAC | 122002 / 00001 | 24.240.049,32 | 24,240,049.32 |

```
Entidade
EMBRATUR    134
Name: count, dtype: int64
```

Out[36]:

| | Mês / Ano de Referência | Entidade | UC/CNPJ | Total Repassado | Valor |
|---|---|---|---|---|---|
| **1027** | jan/20 | EMBRATUR | 35.842.428/0001-66 | 90.202.735,00 | 90,202,735.00 |
| **1044** | fev/20 | EMBRATUR | 35.842.428/0001-66 | 52.274.883,00 | 52,274,883.00 |
| **1061** | mar/20 | EMBRATUR | 35.842.428/0001-66 | 50.715.821,00 | 50,715,821.00 |
| **1078** | abr/20 | EMBRATUR | 35.842.428/0001-66 | 49.549.991,00 | 49,549,991.00 |
| **1095** | mai/20 | EMBRATUR | 35.842.428/0001-66 | 45.933.122,00 | 45,933,122.00 |
| **1112** | jun/20 | EMBRATUR | 35.842.428/0001-66 | 48.627.667,00 | 48,627,667.00 |

In [37]:
```
# Q9 - Cobertura mensal por entidade (após padronizar o nome)
print("Meses distintos na base:", base[COL_DATA].nunique())
cobertura = base.assign(Ent=ent_pad).groupby("Ent")
[COL_DATA].nunique().sort_values()
display(cobertura.to_frame("meses com registro"))
print("Entidades com cobertura diferente do total de meses:",
      (cobertura ≠ base[COL_DATA].nunique()).sum())
```

```
Meses distintos na base: 140
```

| Ent | meses com registro |
|---|---|
| ABDI | 140 |
| ANAC | 140 |
| APEX-BR | 140 |
| DPC | 140 |
| EMBRATUR | 140 |
| FNDE | 140 |
| INCRA | 140 |
| SDR | 140 |
| SEBRAE | 140 |
| SENAC | 140 |
| SENAI | 140 |
| SENAR | 140 |
| SENAT | 140 |
| SESC | 140 |
| SESCOOP | 140 |
| SESI | 140 |
| SEST | 140 |

```
Entidades com cobertura diferente do total de meses: 0
```

In [38]:
```
# Q10 - Possíveis outliers (IQR dentro de cada entidade)
t = base.assign(Entidade=ent_pad, Valor=valor)

iqr_por_ent = t.groupby("Entidade")["Valor"].apply(lambda s:
s.quantile(0.75) - s.quantile(0.25))
print("Entidades com IQR = 0 (método não se aplica):",
list(iqr_por_ent[iqr_por_ent == 0].index))

outlier = t.groupby("Entidade")["Valor"].transform(fora_iqr).astype(bool)
print("Possíveis outliers:", outlier.sum())
display(t[outlier].groupby("Entidade").size().to_frame("outliers"))
display(t.loc[outlier, COL_DATA].str[:3].value_counts().to_frame("mês dos
outliers"))
t[outlier].sort_values("Valor", ascending=False).head(10)
```

```
Entidades com IQR = 0 (método não se aplica): ['EMBRATUR']
Possíveis outliers: 45
```

| Entidade | outliers |
|---|---|
| ABDI | 4 |
| ANAC | 8 |
| APEX-BR | 3 |
| DPC | 2 |
| FNDE | 3 |
| INCRA | 2 |
| SEBRAE | 3 |
| SENAC | 3 |
| SENAI | 4 |
| SENAT | 2 |
| SESC | 3 |
| SESCOOP | 2 |
| SESI | 4 |
| SEST | 2 |

| Mês / Ano de Referência | mês dos outliers |
|---|---|
| jan | 38 |
| jul | 3 |
| ago | 2 |
| mai | 1 |
| nov | 1 |

Out[38]:

| | Mês / Ano de Referência | Entidade | UC/CNPJ | Total Repassado | Valor |
|---|---|---|---|---|---|
| **2244** | jan/26 | FNDE | 153173 / 15253 | 5146437727 | 5,146,437,727.00 |
| **2040** | jan/25 | FNDE | 153173 / 15253 | 4.785.575.662,21 | 4,785,575,662.21 |
| **1836** | jan/24 | FNDE | 153173 / 15253 | 4.322.353.057,00 | 4,322,353,057.00 |
| **2249** | jan/26 | SESC | 33.469.164/0001-11 | 1332041212 | 1,332,041,212.00 |
| **2045** | jan/25 | SESC | 33.469.164/0001-11 | 1.222.040.575,77 | 1,222,040,575.77 |
| **1841** | jan/24 | SESC | 33.469.164/0001-11 | 1.087.903.211,00 | 1,087,903,211.00 |
| **2251** | jan/26 | SEBRAE | 00.330.854/0001-45 | 858527955 | 858,527,955.00 |
| **2046** | jan/25 | SEBRAE | 00.330.854/0001-45 | 798.270.386,80 | 798,270,386.80 |
| **2248** | jan/26 | SENAC | 33.469.172/0001-68 | 774741173,2 | 774,741,173.20 |
| **2044** | jan/25 | SENAC | 33.469.172/0001-68 | 705.641.786,36 | 705,641,786.36 |

In [39]:

```
# 4. LIMPEZA E PREPARAÇÃO (parte de df_raw; df_raw permanece intacto)
df = df_raw.copy()

# D1 (Q1) - Remover linhas totalmente vazias.
# Justificativa: não carregam nenhuma informação (final do arquivo) e não são registros de repasse.
antes = len(df)
df = df.dropna(how="all").reset_index(drop=True)
print("D1 - Linhas vazias removidas:", antes - len(df))
assert df.isna().sum().sum() == 0, "Ainda há nulos parciais: tratar antes de seguir."
```

```
D1 - Linhas vazias removidas: 17
```

In [40]:

```
# D2 (Q2) - Remover linhas idênticas.
# As 16 duplicatas do arquivo bruto eram as linhas vazias (já removidas em D1);
# a chamada abaixo garante que não restem outras.
antes = len(df)
df = df.drop_duplicates().reset_index(drop=True)
print("D2 - Linhas idênticas removidas:", antes - len(df))
```

```
D2 - Linhas idênticas removidas: 0
```

In [41]:
```
# D3 (Q6) - Padronizar entidades.
# - Remover espaços nas pontas ('FNDE ' → 'FNDE').
# - Unificar APEX em APEX-BR: mesmo identificador e períodos que não se
sobrepõem (Q6).
for c in [COL_DATA, COL_ENT, COL_COD, COL_VAL]:
    df[c] = df[c].str.strip()
df[COL_ENT] = df[COL_ENT].replace({"APEX": "APEX-BR"})

print("D3 - Entidades distintas:", df[COL_ENT].nunique())
print("D3 - Duplicados por Mês + Entidade:",
df.duplicated(subset=CHAVE).sum())
df[COL_ENT].value_counts()
```

```
D3 - Entidades distintas: 17
D3 - Duplicados por Mês + Entidade: 0
```

Out[41]:
```
Entidade
FNDE        140
INCRA       140
SENAI       140
SESI        140
SENAC       140
SESC        140
SEBRAE      140
EMBRATUR    140
DPC         140
ANAC        140
SENAR       140
SEST        140
SENAT       140
SESCOOP     140
SDR         140
APEX-BR     140
ABDI        140
Name: count, dtype: int64
```

In [42]:
```
# D4 (Q4) - Converter a data ('jan/15' → 2015-01-01, primeiro dia do mês).
# Os anos têm dois dígitos; assume-se século 21 (dados de 2015 a 2026).
partes = df[COL_DATA].str.split("/", expand=True)
assert partes[0].isin(list(MESES)).all(), "Há mês de referência não
reconhecido."
df[COL_DATA] = pd.to_datetime(
    pd.DataFrame({"year": 2000 + partes[1].astype(int), "month":
partes[0].map(MESES), "day": 1})
)
print("D4 - Período:", df[COL_DATA].min().date(), "→",
df[COL_DATA].max().date())
df[COL_DATA].dtype
```

```
D4 - Período: 2015-01-01 → 2026-08-01
```

Out[42]:
```
dtype('<M8[us]')
```

In [43]:
```
# D5 (Q5) - Converter o valor para número.
# Dois padrões na fonte: até set/25 ponto de milhar + vírgula decimal;
# a partir de out/25 sem milhar, com vírgula decimal e casas truncadas
(perda < R$ 1 por registro).
# Remover todos os pontos é seguro porque Q5 confirmou que nenhum valor do
2º padrão usa ponto.
nulos_antes = df[COL_VAL].isna().sum()
df[COL_VAL] = converte_valor(df[COL_VAL])
assert df[COL_VAL].isna().sum() == nulos_antes, "A conversão gerou novos
nulos."
df[COL_VAL].describe()
```

Out[43]:
```
count           2,380.00
mean      248,313,744.98
std       519,683,030.06
min      -318,467,181.81
25%        19,802,095.50
50%        66,269,871.63
75%       235,194,962.90
max     5,146,437,727.00
Name: Total Repassado, dtype: float64
```

In [44]:
```
# D6 (Q8) - Valor negativo: MANTIDO.
# Há registro(s) negativo(s) (ver Q8). Pode ser estorno ou ajuste contábil, mas isso não é
# confirmado pela fonte e não há como corrigir. Mantido e sinalizado como limitação:
# somas e médias que incluem esse mês são afetadas (tratar no RA2).
negativos = df[df[COL_VAL] < 0]
print("D6 - Registros negativos mantidos:", len(negativos))
negativos
```

```
D6 - Registros negativos mantidos: 1
```

Out[44]:

| | Mês / Ano de Referência | Entidade | UC/CNPJ | Total Repassado |
|---|---|---|---|---|
| **2049** | 2025-01-01 | ANAC | 122002 / 00001 | -318,467,181.81 |

In [45]:
```
# D7 (Q8) - Valores zero: MANTIDOS.
# Zeros concentram-se em uma entidade (ver Q8): ausência de repasse no mês (zero estrutural),
# não dado faltante. Remover distorceria a série temporal dessa entidade.
zeros = df[df[COL_VAL] == 0]
print("D7 - Registros zerados mantidos:", len(zeros))
zeros[COL_ENT].value_counts()
```

```
D7 - Registros zerados mantidos: 134
```

Out[45]:
```
Entidade
EMBRATUR    134
Name: count, dtype: int64
```

In [46]:
```
# D8 (Q10) - Outliers: MANTIDOS.
# Em dados financeiros valores altos costumam ser reais; os outliers concentram-se em janeiro,
# o que sugere sazonalidade e não erro de digitação. Entidades com IQR = 0 não entram no teste.
out_final = df.groupby(COL_ENT)[COL_VAL].transform(fora_iqr).astype(bool)
print("D8 - Possíveis outliers mantidos:", out_final.sum())
print("D8 - Desses, em janeiro:", (df.loc[out_final, COL_DATA].dt.month == 1).sum())
df.loc[out_final].groupby(COL_ENT).size()
```

```
D8 - Possíveis outliers mantidos: 45
D8 - Desses, em janeiro: 38
```

Out[46]:
```
Entidade
ABDI       4
ANAC       8
APEX-BR    3
DPC        2
FNDE       3
INCRA      2
SEBRAE     3
SENAC      3
SENAI      4
SENAT      2
SESC       3
SESCOOP    2
SESI       4
SEST       2
dtype: int64
```

In [47]:
```
# D9 - Ordenar por data e entidade.
df = df.sort_values([COL_DATA, COL_ENT]).reset_index(drop=True)
df.head(17)
```

Out[47]:

| | Mês / Ano de Referência | Entidade | UC/CNPJ | Total Repassado |
|---|---|---|---|---|
| **0** | 2015-01-01 | ABDI | 07.200.966/0001-11 | 8,917,017.00 |
| **1** | 2015-01-01 | ANAC | 122002 / 00001 | 30,221,162.00 |
| **2** | 2015-01-01 | APEX-BR | 05.507.500/0001-38 | 62,241,408.00 |
| **3** | 2015-01-01 | DPC | 852001 / 00001 | 20,988,896.00 |
| **4** | 2015-01-01 | EMBRATUR | 35.842.428/0001-66 | 0.00 |
| **5** | 2015-01-01 | FNDE | 153173 / 15253 | 2,604,077,590.00 |
| **6** | 2015-01-01 | INCRA | 373001 / 37201 | 181,456,016.00 |
| **7** | 2015-01-01 | SDR | 420013 / 00001 | 32,001,111.00 |
| **8** | 2015-01-01 | SEBRAE | 00.330.854/0001-45 | 428,247,534.00 |
| **9** | 2015-01-01 | SENAC | 33.469.172/0001-68 | 334,005,249.00 |
| **10** | 2015-01-01 | SENAI | 33.564.543/0001-90 | 219,119,330.00 |
| **11** | 2015-01-01 | SENAR | 37.138.245/0001-90 | 40,999,819.00 |
| **12** | 2015-01-01 | SENAT | 73.471.963/0001-47 | 37,293,612.00 |
| **13** | 2015-01-01 | SESC | 33.469.164/0001-11 | 597,447,338.00 |
| **14** | 2015-01-01 | SESCOOP | 03.087.543/0001-86 | 39,616,737.00 |
| **15** | 2015-01-01 | SESI | 33.641.358/0001-52 | 313,370,139.00 |
| **16** | 2015-01-01 | SEST | 73.471.989/0001-95 | 57,041,322.00 |

In [48]:
```
# 5. CONJUNTO DE DADOS RESULTANTE
# Comparação antes e depois
comparacao = pd.DataFrame({
    "Medida": ["Registros", "Atributos", "Células nulas", "Linhas idênticas",
               "Nomes distintos de Entidade", "Duplicados por Mês + Entidade (sem vazias)",
               "Tipo da data", "Tipo do valor"],
    "Após ingestão": [
        len(df_raw), df_raw.shape[1], int(df_raw.isna().sum().sum()), int(df_raw.duplicated().sum()),
        df_raw[COL_ENT].nunique(), int(base.duplicated(subset=CHAVE).sum()),
        str(df_raw[COL_DATA].dtype), str(df_raw[COL_VAL].dtype)],
    "Após preparação": [
        len(df), df.shape[1], int(df.isna().sum().sum()), int(df.duplicated().sum()),
        df[COL_ENT].nunique(), int(df.duplicated(subset=CHAVE).sum()),
        str(df[COL_DATA].dtype), str(df[COL_VAL].dtype)],
})
print(f"Perda de registros: {(1 - len(df) / len(df_raw)) * 100:.2f}% (linhas vazias)")
comparacao
```

```
Perda de registros: 0.71% (linhas vazias)
```

Out[48]:

| | Medida | Após ingestão | Após preparação |
|---|---|---|---|
| **0** | Registros | 2397 | 2380 |
| **1** | Atributos | 4 | 4 |
| **2** | Células nulas | 68 | 0 |
| **3** | Linhas idênticas | 16 | 0 |
| **4** | Nomes distintos de Entidade | 19 | 17 |
| **5** | Duplicados por Mês + Entidade (sem vazias) | 0 | 0 |
| **6** | Tipo da data | str | datetime64[us] |
| **7** | Tipo do valor | str | float64 |

In [49]:
```
# Verificações finais
assert df.isna().sum().sum() == 0
assert not df.duplicated().any()
assert not df.duplicated(subset=CHAVE).any()
assert df[COL_ENT].nunique() == 17
assert df.groupby(COL_ENT).size().eq(140).all()
assert df.groupby(COL_ENT)[COL_COD].nunique().eq(1).all()
assert df[COL_DATA].nunique() == 140
assert pd.api.types.is_datetime64_any_dtype(df[COL_DATA])
assert pd.api.types.is_float_dtype(df[COL_VAL])

print("Registros:", len(df))
print("Entidades:", df[COL_ENT].nunique())
print("Período:", df[COL_DATA].min().strftime("%m/%Y"), "a",
df[COL_DATA].max().strftime("%m/%Y"))
print("Negativos mantidos:", (df[COL_VAL] < 0).sum(), "| zeros mantidos:",
(df[COL_VAL] == 0).sum())
df.info()
```

```
Registros: 2380
Entidades: 17
Período: 01/2015 a 08/2026
Negativos mantidos: 1 | zeros mantidos: 134
<class 'pandas.DataFrame'>
RangeIndex: 2380 entries, 0 to 2379
Data columns (total 4 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Mês / Ano de Referência  2380 non-null   datetime64[us]
 1   Entidade                2380 non-null   str
 2   UC/CNPJ                 2380 non-null   str
 3   Total Repassado         2380 non-null   float64
dtypes: datetime64[us](1), float64(1), str(2)
memory usage: 74.5 KB
```

In [50]:
```
# Gravação do conjunto final e releitura para conferir o arquivo salvo
df.to_csv(PROCESSED, sep=";", index=False)

check = pd.read_csv(PROCESSED, sep=";", parse_dates=[COL_DATA], dtype=
{COL_COD: str})
print("Salvo em:", PROCESSED)
print("Shape salvo:", check.shape, "| igual ao df:", check.shape ==
df.shape)
print("Valores iguais:", bool((check[COL_VAL].round(2) ==
df[COL_VAL].round(2)).all()))
check.head()
```

```
Salvo em: ..\data\processed\repasses_arrecadacao_federal_limpo.csv
Shape salvo: (2380, 4) | igual ao df: True
Valores iguais: True
```

Out[50]:

| | Mês / Ano de Referência | Entidade | UC/CNPJ | Total Repassado |
|---|---|---|---|---|
| **0** | 2015-01-01 | ABDI | 07.200.966/0001-11 | 8,917,017.00 |
| **1** | 2015-01-01 | ANAC | 122002 / 00001 | 30,221,162.00 |
| **2** | 2015-01-01 | APEX-BR | 05.507.500/0001-38 | 62,241,408.00 |
| **3** | 2015-01-01 | DPC | 852001 / 00001 | 20,988,896.00 |
| **4** | 2015-01-01 | EMBRATUR | 35.842.428/0001-66 | 0.00 |